



Iterative Methods for solving linear systems

Dr. Helene Kanso

Solving linear systems

- ▶ We have discussed solving linear systems using Direct numerical methods like
 - Gaussian elimination
 - LU factorization
 - Matrix inversion
 - Other methods exist too

Why to use iterative methods?

- Direct methods are not ideal for solving **very large system** of linear equations
- Direct methods do not work for **non-linear equations**
- Iterative methods are more efficient when dealing with large systems and **sparse matrices**
- Iterative methods are useful when an **approximate solution** is needed rather than an exact one

What are iterative methods?

- Consider the following 3x3 linear systems (easily generalized to nxn):

$$\begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

- Can be written as:

$$\begin{bmatrix} a_{1,1} * x_1 + a_{1,2} * x_2 + a_{1,3} * x_3 \\ a_{2,1} * x_1 + a_{2,2} * x_2 + a_{2,3} * x_3 \\ a_{3,1} * x_1 + a_{3,2} * x_2 + a_{3,3} * x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

What are iterative methods?

➡ Or as:

$$\begin{bmatrix} \sum_{j=1}^n a_{1,j} * x_j \\ \sum_{j=1}^n a_{2,j} * x_j \\ \sum_{j=1}^n a_{3,j} * x_j \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

What are iterative methods?

- ▶ In general, for a row i :

$$\sum_{j=1}^n a_{i,j} * x_j = b_i$$

- ▶ Our goal is to find some x -values that will give us the proper b -values!

What are iterative methods?

- Let's re-write the equation by taking $a_{i,i} * x_i$ out of the summation:

$$a_{i,i} * x_i + \sum_{j=1, j \neq i}^n a_{i,j} * x_j = b_i$$

- Let's isolate now x_i , we obtain:

$$x_i = \frac{b_i - \sum_{j=1, j \neq i}^n a_{i,j} * x_j}{a_{i,i}}$$

What are iterative methods?

- ▶ We have no way of solving this problem as all x-values are unknown
- ▶ WHY NOT TO GUESS THEN?!
- ▶ k will represent an iteration

$$x_i^{(k+1)} = \frac{b_i - \sum_{j=1, j \neq i}^n a_{i,j} * x_j^{(k)}}{a_{i,i}}$$

What are iterative methods?


$$x_i^{(k+1)} = \frac{b_i - \sum_{j=1, j \neq i}^n a_{i,j} * x_j^{(k)}}{a_{i,i}}$$


$$x_i^{(k+1)} = \underbrace{x_i^{(k)} - x_i^{(k)}}_0 + \frac{b_i - \sum_{j=1, j \neq i}^n a_{i,j} * x_j^{(k)}}{a_{i,i}}$$


$$x_i^{(k+1)} = x_i^{(k)} + \frac{b_i - \sum_{j=1}^n a_{i,j} * x_j^{(k)}}{a_{i,i}}$$

What are iterative methods?


$$x_i^{(k+1)} = x_i^{(k)} + \frac{b_i - A_i x^{(k)}}{a_{i,i}}$$

$$b_i - A_i x^{(k)}$$

Is a “remainder term” which tells us HOW FAR we are from the real x-values.

Our goal is to turn this **ZERO**

What is convergence?

- Convergence means that after a sufficiently large number of iterations, the results of the iterative methods get closer and closer to the true solution.
- We can never be certain of the number of iterations that are required
- Good initial guesses (at iteration $k=0$) can reduce the number of needed iterations
- Convergence is guaranteed if the matrix is **diagonally dominant**

What is a diagonally dominant matrix?

$$|a_{i,i}| \geq \sum_{j=1, j \neq i}^n |a_{i,j}| \text{ for all rows}$$

AND

$$|a_{i,i}| > \sum_{j=1, j \neq i}^n |a_{i,j}| \text{ for at least one row}$$

Example:



$$\begin{bmatrix} 9 & 2 & 4 \\ 4 & 7 & 1 \\ 3 & 3 & 7 \end{bmatrix}$$

is diagonally dominant



$$\begin{bmatrix} 4 & 1 & 1 & 1 \\ 1 & 4 & 2 & 3 \\ 6 & 7 & 3 & 1 \\ 9 & 9 & 8 & 10 \end{bmatrix}$$

is not diagonally dominant

Popular iterative methods

- Jacobi Method
- Gauss-Seidel
- Successive Over-relaxation
- Conjugate Gradient
- GMRES

Jacobi method

Step 1: Make initial guesses:

Typically $x_i^{(0)} = 0$, for any row i

Step 2:

While “Remainder” > tolerance (acceptable error)
do Step 3

Step 3: For each row i do:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j=1, j \neq i}^n a_{i,j} * x_j^{(k)}}{a_{i,i}}$$

Jacobi method (example)

16

$$5x + 2y = 16$$

$$x + 9y = 11$$

$$x_{k+1} = \frac{1}{5}(16 - 2y_k)$$

$$y_{k+1} = \frac{1}{9}(11 - x_k)$$

Initial guess: $x_0 = 0, y_0 = 0$

1st Approximation

$$x_1 = \frac{1}{5}[16 - 2(0)] = \frac{1}{5}[16] = 3.2$$

$$y_1 = \frac{1}{9}[11 - (0)] = \frac{1}{9}[11] = 1.2222$$

Jacobi method (example)



2nd Approximation

$$x_2 = \frac{1}{5}[16 - 2(1.2222)] = \frac{1}{5}[13.5556] = 2.7111$$

$$y_2 = \frac{1}{9}[11 - (3.2)] = \frac{1}{9}[7.8] = 0.8667$$


3rd Approximation

$$x_3 = \frac{1}{5}[16 - 2(0.8667)] = \frac{1}{5}[14.2667] = 2.8533$$

$$y_3 = \frac{1}{9}[11 - (2.7111)] = \frac{1}{9}[8.2889] = 0.921$$

4th Approximation

$$x_4 = \frac{1}{5}[16 - 2(0.921)] = \frac{1}{5}[14.158] = 2.8316$$

$$y_4 = \frac{1}{9}[11 - (2.8533)] = \frac{1}{9}[8.1467] = 0.9052$$


5th Approximation

$$x_5 = \frac{1}{5}[16 - 2(0.9052)] = \frac{1}{5}[14.1896] = 2.8379$$

$$y_5 = \frac{1}{9}[11 - (2.8316)] = \frac{1}{9}[8.1684] = 0.9076$$

Jacobi method (example)

6th Approximation

$$x_6 = \frac{1}{5}[16 - 2(0.9076)] = \frac{1}{5}[14.1848] = 2.837$$

$$y_6 = \frac{1}{9}[11 - (2.8379)] = \frac{1}{9}[8.1621] = 0.9069$$

Iterations are tabulated as below

Iteration	x	y
1	3.2	1.2222
2	2.7111	0.8667
3	2.8533	0.921
4	2.8316	0.9052
5	2.8379	0.9076
6	2.837	0.9069

Gauss-Seidel method

- Very similar to Jacobi method!
- Gauss-Seidel updates variables sequentially while using the most recent values for each variable during the same iteration
- Convergence is **generally quicker** than Jacobi for diagonally dominant or positive-definite matrices and requires **less memory** than Jacobi

Gauss-Seidel method

Step 1: Make initial guesses:

Typically $x_i^{(0)} = 0$, for any row i

Step 2:

While “Remainder” > tolerance (acceptable error)
do Step 3

Step 3: For each row i do:

Main difference between
Jacobi and Gauss-Seidel

$$x_i^{(k+1)} = \frac{b_i - \sum_{j=1, j \neq i}^n a_{i,j} * x_j^{(most\ recent\ iter)}}{a_{i,i}}$$

Gauss-Seidel method (example)

21

$$5x + 2y = 16$$

$$x + 9y = 11$$

$$x_{k+1} = \frac{1}{5}(16 - 2y_k)$$

$$y_{k+1} = \frac{1}{9}(11 - x_{k+1})$$

Initial guess: $x_0 = 0, y_0 = 0$

1st Approximation

$$x_1 = \frac{1}{5}[16 - 2(0)] = \frac{1}{5}[16] = 3.2$$

$$y_1 = \frac{1}{9}[11 - (3.2)] = \frac{1}{9}[7.8] = 0.8667$$

Gauss-Seidel method (example 2)

2nd Approximation

$$x_2 = \frac{1}{5}[16 - 2(0.8667)] = \frac{1}{5}[14.2667] = 2.8533$$

$$y_2 = \frac{1}{9}[11 - (2.8533)] = \frac{1}{9}[8.1467] = 0.9052$$

3rd Approximation

$$x_3 = \frac{1}{5}[16 - 2(0.9052)] = \frac{1}{5}[14.1896] = 2.8379$$

$$y_3 = \frac{1}{9}[11 - (2.8379)] = \frac{1}{9}[8.1621] = 0.9069$$

4th Approximation

$$x_4 = \frac{1}{5}[16 - 2(0.9069)] = \frac{1}{5}[14.1862] = 2.8372$$

$$y_4 = \frac{1}{9}[11 - (2.8372)] = \frac{1}{9}[8.1628] = 0.907$$

Iterations are tabulated as below

Iteration	x	y
1	3.2	0.8667
2	2.8533	0.9052
3	2.8379	0.9069
4	2.8372	0.907

Successive over relaxation method

- Based on a relaxation factor ω
- $x_i^{(k+1)}$ is calculated as the weighted average of the previous estimate $x_i^{(k)}$ and the estimate using Gauss-Seidel $\left(x_i^{(k+1)}\right)_{Gauss-Seidel}$

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \omega \left(x_i^{(k+1)}\right)_{Gauss-Seidel}$$

- $\omega = 1$: Gauss-Seidel
- $0 < \omega < 1$: Under-relaxation, helps stabilize convergence but slows the process
- $1 < \omega < 2$: Over-relaxation, helps speed up convergence (ω shouldn't be too large though)

Successive over relaxation method

- ω is a tuning parameter that allows for more flexibility in accelerating convergence (fine tuning)

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \omega \left(x_i^{(k+1)} \right)_{Gauss-Seidel}$$

Beyond basic methods

- Krylov subspace methods (like Conjugate Gradient **CG** method and Generalized Minimal Residual **GMRES**)
- Fast Fourier transform
- Block cyclic reduction
- Multigrid method